

**Федеральное государственное автономное образовательное учреждение
высшего образования**

"Национальный исследовательский университет

"Высшая школа экономики"

Московский институт электроники и математики им. А. Н. Тихонова

Департамент компьютерной инженерии

Отчёт по дисциплине

«Распределённые базы данных и сетевые вычисления»

Система ввода/вывода. Поток данных (stream). Исключения

Студент:

Абрамов Илья Александрович

Группа: МКС254

Преподаватель: Байбикова Т.Н.

1 Задание

Добавить в разработанные в предыдущих работах классах возможность записи данных в файл и считывания данных из файла.

Подготовьте отчет. В отчет включите:

- задание;
- разработанную программу или набор программ (укажите в листингах, где ввод-вывод);
- опишите в отчете, какие использовали классы для ввода-вывода;
- опишите в отчете, какие использовали классы для обработки исключений;
- результаты работы программы;
- выводы.

2 Разработанная программа

Для реализации файловых операций была разработана следующая архитектура:

- FileManager - утилитный класс для работы с файлами
- FileOperationException - пользовательское исключение для файловых операций
- Модифицированный App - демонстрация работы с файлами

Для работы с файлами были использованы следующие классы из пакета java.io:

- FileReader - для чтения символов из файла
- FileWriter - для записи символов в файл
- BufferedReader - для буферизованного чтения из файла
- BufferedWriter - для буферизованной записи в файл
- IOException - базовый класс исключений ввода-вывода
- FileNotFoundException - исключение при отсутствии файла

Для обработки исключений были использованы:

- Стандартные исключения Java:
 - IOException - ошибки ввода/вывода
 - FileNotFoundException - файл не найден
- Пользовательское исключение:
 - FileOperationException - создано для инкапсуляции ошибок файловых операций

3 Описание разработанных классов

FileOperationException.java - пользовательское исключение для обработки ошибок файловых операций.

FileManager.java - утилитный класс, предоставляющий методы для работы с файлами:

- saveToFile(Object obj, String filePath) - сохранение объекта в JSON файл
- loadFromFile(String filePath, Class<T> classType) - загрузка объекта из JSON файла
- saveCollectionToFile(List<T> collection, String filePath) - сохранение коллекции в JSON файл
- loadCollectionFromFile(String filePath, Class<T> classType) - загрузка коллекции из JSON файла
- objectToJson(Object obj) - преобразование объекта в JSON строку
- jsonToObject(String json, Class<T> classType) - преобразование JSON в объект
- collectionToJson(List<T> collection) - преобразование коллекции в JSON
- jsonToCollection(String json, Class<T> classType) - преобразование JSON в коллекцию
- createDirectoryIfNotExists(String filePath) - создание директории при необходимости

4 Листинг

4.1 Файл FileOperationException.java

```
public class FileOperationException extends Exception {  
    public FileOperationException(String message) {  
        super(message);  
    }  
  
    public FileOperationException(String message, Throwable cause) {  
        super(message, cause);  
    }  
}
```

4.2 Файл FileManager.java

```
import java.io.*;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

// Утилитный класс для работы с файлами
public class FileManager {

    // Сохранение объекта в JSON файл
    public static void saveToFile(Object obj, String filePath) throws
    FileOperationException {
        try {
            createDirectoryIfNotExists(filePath);
            String json = objectToJson(obj);

            try (BufferedWriter writer = new BufferedWriter(new
    FileWriter(filePath))) {
                writer.write(json);
            }
        } catch (IOException e) {
            throw new FileOperationException("Ошибка при сохранении объекта в
    файл: " + filePath, e);
        }
    }

    // Загрузка объекта из JSON файла
    public static <T> T loadFromFile(String filePath, Class<T> classType) throws
    FileOperationException {
        try (BufferedReader reader = new BufferedReader(new
    FileReader(filePath))) {
            StringBuilder jsonBuilder = new StringBuilder();
            String line;
            while ((line = reader.readLine()) != null) {
                jsonBuilder.append(line);
            }

            String json = jsonBuilder.toString();
            return jsonToObject(json, classType);
        } catch (FileNotFoundException e) {
            throw new FileOperationException("Файл не найден: " + filePath, e);
        } catch (IOException e) {
            throw new FileOperationException("Ошибка при чтении из файла: " +
    filePath, e);
        }
    }

    // Сохранение коллекции объектов в JSON файл
    public static <T> void saveCollectionToFile(List<T> collection, String
    filePath) throws FileOperationException {
```

```

        try {
            createDirectoryIfNotExists(filePath);
            String json = collectionToJson(collection);

            try (BufferedWriter writer = new BufferedWriter(new
FileWriter(filePath))) {
                writer.write(json);
            }
        } catch (IOException e) {
            throw new FileOperationException("Ошибка при сохранении коллекции в
файл: " + filePath, e);
        }
    }

    // Загрузка коллекции объектов из JSON файла
    public static <T> List<T> loadCollectionFromFile(String filePath, Class<T>
classType) throws FileOperationException {
        try (BufferedReader reader = new BufferedReader(new
FileReader(filePath))) {
            StringBuilder jsonBuilder = new StringBuilder();
            String line;
            while ((line = reader.readLine()) != null) {
                jsonBuilder.append(line);
            }

            String json = jsonBuilder.toString();
            return jsonToCollection(json, classType);
        } catch (FileNotFoundException e) {
            throw new FileOperationException("Файл не найден: " + filePath, e);
        } catch (IOException e) {
            throw new FileOperationException("Ошибка при чтении из файла: " +
filePath, e);
        }
    }

    // Преобразование объекта в JSON строку (упрощенная реализация)
    private static String objectToJson(Object obj) {
        if (obj == null) return "null";

        StringBuilder json = new StringBuilder();
        json.append("{");

        Field[] fields = obj.getClass().getDeclaredFields();
        boolean first = true;

        for (Field field : fields) {
            field.setAccessible(true);
            try {
                Object value = field.get(obj);
                if (!first) {
                    json.append(",");
                }
            }

```

```

    }
    json.append("\").append(field.getName()).append("\":");
    json.append(valueToJson(value));
    first = false;
} catch (IllegalAccessException e) {
    // Пропускаем поля, к которым нет доступа
}

}

// Добавляем поля из родительских классов
Class<?> superClass = obj.getClass().getSuperclass();
while (superClass != null && superClass != Object.class) {
    Field[] superFields = superClass.getDeclaredFields();
    for (Field field : superFields) {
        field.setAccessible(true);
        try {
            Object value = field.get(obj);
            if (!first) {
                json.append(",");
            }
            json.append("\").append(field.getName()).append("\":");
            json.append(valueToJson(value));
            first = false;
        } catch (IllegalAccessException e) {
            // Пропускаем поля, к которым нет доступа
        }
    }
    superClass = superClass.getSuperclass();
}

json.append("}");
return json.toString();
}

// Преобразование значения в JSON формат
private static String valueToJson(Object value) {
    if (value == null) return "null";
    if (value instanceof String) return "\"" + value + "\"";
    if (value instanceof Number || value instanceof Boolean) return
value.toString();
    if (value instanceof Character) return "\"" + value + "\"";

    // Для сложных объектов рекурсивно вызываем objectToJson
    return objectToJson(value);
}

// Преобразование JSON строки в объект (упрощенная реализация)
private static <T> T jsonToObject(String json, Class<T> classType) throws
FileNotFoundException {
    try {
        T obj = classType.getDeclaredConstructor().newInstance();

```



```

        // Упрощенный парсинг JSON (в реальном проекте лучше использовать
библиотеку)
        json = json.trim();
        if (json.startsWith("{") && json.endsWith("}")) {
            json = json.substring(1, json.length() - 1);

            String[] pairs = json.split(",");
            for (String pair : pairs) {
                String[] keyValue = pair.split(":", 2);
                if (keyValue.length == 2) {
                    String key = keyValue[0].trim().replace("\\\"", "");
                    String value = keyValue[1].trim();

                    try {
                        Field field = findField(obj.getClass(), key);
                        if (field != null) {
                            field.setAccessible(true);
                            field.set(obj, parseValue(value,
field.getType()));
                        }
                    } catch (Exception e) {
                        // Пропускаем поля, которые не удалось установить
                    }
                }
            }
        }

        return obj;
    } catch (Exception e) {
        throw new FileOperationException("Ошибка при преобразовании JSON в
объект", e);
    }
}

// Поиск поля в классе и его родительских классах
private static Field findField(Class<?> clazz, String fieldName) {
    while (clazz != null && clazz != Object.class) {
        try {
            return clazz.getDeclaredField(fieldName);
        } catch (NoSuchFieldException e) {
            clazz = clazz.getSuperclass();
        }
    }
    return null;
}

// Преобразование строкового значения в нужный тип
private static Object parseValue(String value, Class<?> targetType) {
    value = value.trim();

```

```

        if (value.equals("null")) return null;
        if (value.startsWith("\"") && value.endsWith("\"")) {
            return value.substring(1, value.length() - 1);
        }

        if (targetType == int.class || targetType == Integer.class) {
            return Integer.parseInt(value);
        }
        if (targetType == double.class || targetType == Double.class) {
            return Double.parseDouble(value);
        }
        if (targetType == boolean.class || targetType == Boolean.class) {
            return Boolean.parseBoolean(value);
        }
        if (targetType == long.class || targetType == Long.class) {
            return Long.parseLong(value);
        }
        if (targetType == float.class || targetType == Float.class) {
            return Float.parseFloat(value);
        }
        if (targetType == char.class || targetType == Character.class) {
            if (value.length() >= 3 && value.startsWith("\"") &&
value.endsWith("\"")) {
                return value.charAt(1);
            }
        }

        return value;
    }

    // Преобразование коллекции в JSON
    private static <T> String collectionToJson(List<T> collection) {
        StringBuilder json = new StringBuilder();
        json.append("[");

        for (int i = 0; i < collection.size(); i++) {
            if (i > 0) json.append(",");
            json.append(objectToJson(collection.get(i)));
        }

        json.append("]");
        return json.toString();
    }

    // Преобразование JSON в коллекцию
    private static <T> List<T> jsonToCollection(String json, Class<T> classType)
throws FileNotFoundException {
        List<T> collection = new ArrayList<>();

        json = json.trim();
        if (json.startsWith("[") && json.endsWith("]")) {

```

```

        json = json.substring(1, json.length() - 1);

        // Упрощенный парсинг массива JSON
        int depth = 0;
        int start = 0;

        for (int i = 0; i < json.length(); i++) {
            char c = json.charAt(i);
            if (c == '{') depth++;
            else if (c == '}') depth--;
            else if (c == ',' && depth == 0) {
                String itemJson = json.substring(start, i).trim();
                if (!itemJson.isEmpty()) {
                    collection.add(jsonObject(itemJson, classType));
                }
                start = i + 1;
            }
        }

        // Добавляем последний элемент
        if (start < json.length()) {
            String itemJson = json.substring(start).trim();
            if (!itemJson.isEmpty()) {
                collection.add(jsonObject(itemJson, classType));
            }
        }

        return collection;
    }

    // Создание директории, если она не существует
    private static void createDirectoryIfNotExists(String filePath) throws
IOException {
        File file = new File(filePath);
        File parentDir = file.getParentFile();

        if (parentDir != null && !parentDir.exists()) {
            if (!parentDir.mkdirs()) {
                throw new IOException("Не удалось создать директорию: " +
parentDir.getAbsolutePath());
            }
        }
    }
}

```

4.3 Файл Engine.java

```
public class Engine {
    private String model;
    private int horsepower;
    private double fuelConsumptionRate;

    // ПЕРЕГРУЗКА КОНСТРУКТОРОВ
    public Engine() {
        this("Unknown", 1000, 5.0);
    }

    public Engine(String model) {
        this(model, 1000, 5.0);
    }

    public Engine(String model, int horsepower) {
        this(model, horsepower, 5.0);
    }

    public Engine(String model, int horsepower, double fuelConsumptionRate) {
        this.model = model;
        this.horsePower = horsepower;
        this.fuelConsumptionRate = fuelConsumptionRate;
    }

    // ПЕРЕГРУЗКА МЕТОДОВ
    public void start() {
        System.out.println("Engine " + model + " starting...");
    }

    public void start(boolean quickStart) {
        if (quickStart) {
            System.out.println("Engine " + model + " quick starting!");
        } else {
            start();
        }
    }

    public void stop() {
        System.out.println("Engine " + model + " stopped.");
    }

    public String getInfo() {
        return "Engine: " + model + ", HP: " + horsepower + ", Consumption: " +
fuelConsumptionRate + " L/h";
    }

    public double getFuelConsumptionRate() {
        return fuelConsumptionRate;
    }
}
```

```
public int getHorsePower() {  
    return horsePower;  
}  
}
```

4.4 Файл App.java (добавленный функционал)

```
private static void demonstrateFileOperations() {
    try {
        // Создаем тестовые объекты
        System.out.println("\n--- Создание тестовых объектов ---");
        Engine testEngine = new Engine("Test Engine", 30000, 15.5);
        PassengerPlane testPassengerPlane = new PassengerPlane("TestAir",
"Test-100", 2023, 950.0,
                                                    200, 40.0,
testEngine,
                                                    30, 170,
true);
        CargoPlane testCargoPlane = new CargoPlane("CargoAir", "Cargo-200",
2022, 900.0,
                                                    45.0, testEngine,
                                                    100000.0, 750.0, true);

        // Демонстрация сохранения отдельных объектов
        System.out.println("\n--- Сохранение отдельных объектов в файлы ---");

        String engineFile = "data/engine.json";
        String passengerPlaneFile = "data/passenger_plane.json";
        String cargoPlaneFile = "data/cargo_plane.json";

        FileManager.saveToFile(testEngine, engineFile);
        System.out.println("✓ Двигатель сохранен в файл: " + engineFile);

        FileManager.saveToFile(testPassengerPlane, passengerPlaneFile);
        System.out.println("✓ Пассажирский самолет сохранен в файл: " +
passengerPlaneFile);

        FileManager.saveToFile(testCargoPlane, cargoPlaneFile);
        System.out.println("✓ Грузовой самолет сохранен в файл: " +
cargoPlaneFile);

        // Демонстрация загрузки отдельных объектов
        System.out.println("\n--- Загрузка отдельных объектов из файлов ---");

        Engine loadedEngine = FileManager.loadFromFile(engineFile,
Engine.class);
        System.out.println("✓ Двигатель загружен из файла");
        System.out.println("  Оригинал: " + testEngine.getInfo());
        System.out.println("  Загружен: " + loadedEngine.getInfo());

        PassengerPlane loadedPassengerPlane =
FileManager.loadFromFile(passengerPlaneFile, PassengerPlane.class);
        System.out.println("✓ Пассажирский самолет загружен из файла");
        System.out.println("  Оригинал: " +
testPassengerPlane.getManufacturer() + " " + testPassengerPlane.getModel());
```

```

        System.out.println("    Загружен: " +
loadedPassengerPlane.getManufacturer() + " " + loadedPassengerPlane.getModel());

        CargoPlane loadedCargoPlane =
FileManager.loadFromFile(cargoPlaneFile, CargoPlane.class);
        System.out.println("✓ Грузовой самолет загружен из файла");
        System.out.println("    Оригинал: " + testCargoPlane.getManufacturer()
+ " " + testCargoPlane.getModel());
        System.out.println("    Загружен: " +
loadedCargoPlane.getManufacturer() + " " + loadedCargoPlane.getModel());

        // Демонстрация работы с коллекциями
        System.out.println("\n--- Работа с коллекциями объектов ---");
        java.util.ArrayList<Aeroplane> fleet = new java.util.ArrayList<>();

        // Создаем дополнительные объекты для коллекции
        Engine airbusEngine = new Engine("Rolls-Royce Trent 900", 36300,
11.8);
        PassengerPlane airbus380 = new PassengerPlane("Airbus", "A380",
2021, 945.0,
                                                    853, 79.8,
airbusEngine,
                                                    100, 753, true);

        Engine cargoEngine = new Engine("GE CF6-80C2", 28000, 10.5);
        CargoPlane boeing747F = new CargoPlane("Boeing", "747-8F", 2019,
908.0,
                                                    68.4, cargoEngine,
                                                    140000.0, 858.0, true);

        fleet.add(airbus380);
        fleet.add(boeing747F);
        fleet.add(testPassengerPlane);
        fleet.add(testCargoPlane);

        String fleetFile = "data/fleet.json";
        FileManager.saveCollectionToFile(fleet, fleetFile);
        System.out.println("✓ Коллекция самолетов сохранена в файл: " +
fleetFile);

        java.util.List<Aeroplane> loadedFleet =
FileManager.loadCollectionFromFile(fleetFile, Aeroplane.class);
        System.out.println("✓ Коллекция самолетов загружена из файла");
        System.out.println("    Размер оригинальной коллекции: " +
fleet.size());
        System.out.println("    Размер загруженной коллекции: " +
loadedFleet.size());

        // Демонстрация обработки исключений
        System.out.println("\n--- Демонстрация обработки исключений ---");
        try {

```

```

        FileManager.loadFromFile("nonexistent_file.json", Engine.class);
    } catch (FileOperationException e) {
        System.out.println("✓ Перехвачено исключение: " +
e.getMessage());
    }

    System.out.println("\n" + "=".repeat(70));
    System.out.println("РАБОТА С ФАЙЛАМИ УСПЕШНО ЗАВЕРШЕНА");
    System.out.println("=".repeat(70));

    } catch (FileOperationException e) {
        System.err.println("Ошибка при работе с файлами: " +
e.getMessage());
        e.printStackTrace();
    }
}

```


5 Результат работы программы

```
=====
11. ДЕМОНСТРАЦИЯ РАБОТЫ С ФАЙЛАМИ
=====
```

```
--- Создание тестовых объектов ---
```

```
--- Сохранение отдельных объектов в файлы ---
```

```
? Двигатель сохранен в файл: data/engine.json
```

```
? Пассажирский самолет сохранен в файл: data/passenger_plane.json
```

```
? Грузовой самолет сохранен в файл: data/cargo_plane.json
```

```
--- Загрузка отдельных объектов из файлов ---
```

```
? Двигатель загружен из файла
```

```
Оригинал: Engine: Test Engine, HP: 30000, Consumption: 15.5 L/h
```

```
Загружен: Engine: Test Engine, HP: 30000, Consumption: 15.5 L/h
```

```
? Пассажирский самолет загружен из файла
```

```
Оригинал: TestAir Test-100
```

```
Загружен: TestAir Test-100
```

```
? Грузовой самолет загружен из файла
```

```
Оригинал: CargoAir Cargo-200
```

```
Загружен: CargoAir Cargo-200
```

```
--- Работа с коллекциями объектов ---
```

```
? Коллекция самолетов сохранена в файл: data/fleet.json
```

```
? Коллекция самолетов загружена из файла
```

```
Размер оригинальной коллекции: 4
```

```
Размер загруженной коллекции: 4
```

```
--- Демонстрация обработки исключений ---
```

```
? Перехвачено исключение: Файл не найден: nonexistent_file.json
```

```
=====
РАБОТА С ФАЙЛАМИ УСПЕШНО ЗАВЕРШЕНА
=====
```

6 Сохраненные файлы

6.1 CargoPlane.json

```
{
  "maxCargoWeight":100000.0,
  "cargoVolumeCapacity":750.0,
  "hasLoadingRamp":true,
  "passengerCapacity":0,
  "wingspan":45.0,
  "engine":{
    "model":"Test Engine",
    "horsePower":30000,
    "fuelConsumptionRate":15.5
  },
  "flightHours":0,
  "isFlying":false,
  "manufacturer":"CargoAir",
  "model":"Cargo-200",
  "year":2022,
  "maxSpeed":900.0
}
```

6.2 Engine.json

```
{
  "model":"Test Engine",
  "horsePower":30000,
  "fuelConsumptionRate":15.5
}
```

6.3 Fleet.json

```
[
  {
    "businessClassSeats":100,
    "economyClassSeats":753,
    "hasWiFi":true,
    "passengerCapacity":853,
    "wingspan":79.8,
    "engine":{
      "model":"Rolls-Royce Trent 900",
      "horsePower":36300,
      "fuelConsumptionRate":11.8
    },
    "flightHours":0,
    "isFlying":false,
    "manufacturer":"Airbus",
    "model":"A380",
    "year":2021,
  }
]
```

```

    "maxSpeed":945.0
  },
  {
    "maxCargoWeight":140000.0,
    "cargoVolumeCapacity":858.0,
    "hasLoadingRamp":true,
    "passengerCapacity":0,
    "wingspan":68.4,
    "engine":{
      "model":"GE CF6-80C2",
      "horsePower":28000,
      "fuelConsumptionRate":10.5
    },
    "flightHours":0,
    "isFlying":false,
    "manufacturer":"Boeing",
    "model":"747-8F",
    "year":2019,
    "maxSpeed":908.0
  },
  {
    "businessClassSeats":30,
    "economyClassSeats":170,
    "hasWiFi":true,
    "passengerCapacity":200,
    "wingspan":40.0,
    "engine":{
      "model":"Test Engine",
      "horsePower":30000,
      "fuelConsumptionRate":15.5
    },
    "flightHours":0,
    "isFlying":false,
    "manufacturer":"TestAir",
    "model":"Test-100",
    "year":2023,
    "maxSpeed":950.0
  },
  {
    "maxCargoWeight":100000.0,
    "cargoVolumeCapacity":750.0,
    "hasLoadingRamp":true,
    "passengerCapacity":0,
    "wingspan":45.0,
    "engine":{
      "model":"Test Engine",
      "horsePower":30000,

```

```

        "fuelConsumptionRate":15.5
    },
    "flightHours":0,
    "isFlying":false,
    "manufacturer":"CargoAir",
    "model":"Cargo-200",
    "year":2022,
    "maxSpeed":900.0
}
]

```

6.4 PassengerPlane.json

```

{
    "businessClassSeats":30,
    "economyClassSeats":170,
    "hasWiFi":true,
    "passengerCapacity":200,
    "wingspan":40.0,
    "engine":{
        "model":"Test Engine",
        "horsePower":30000,
        "fuelConsumptionRate":15.5
    },
    "flightHours":0,
    "isFlying":false,
    "manufacturer":"TestAir",
    "model":"Test-100",
    "year":2023,
    "maxSpeed":950.0
}

```

Выводы

В результате выполнения лабораторной работы были освоены основы работы с файлами в Java, включая чтение и запись данных с использованием классов `FileReader`, `FileWriter`, `BufferedReader`, `BufferedWriter`. Реализована возможность сохранения объектов в структурированном JSON-формате и восстановления их из файлов. Показана работа с различными типами данных: примитивами, строками, объектами и коллекциями. В процессе работы были применены принципы обработки исключений, включая создание пользовательского исключения и обработку стандартных исключений ввода-вывода. Полученный код обеспечивает надежное и надежное взаимодействие с файловой системой, сохраняя целостность данных и предоставляя удобный интерфейс для операций с файлами.